10 Práce s polem, práce s datem a časem

V této kapitole se dozvíte:

- Jak vytvořit pole a procházet jeho prvky.
- Jak vyhledávat v poli a řadit pole.
- Jak pracovat s datem a časem pomocí funkcí a pomocí třídy DateTime.

Po jejím prostudování byste měli být schopni:

- Umět použít funkce pro vytvoření a procházení polí.
- Umět použít funkce pro vyhledávání v poli a pro řazení polí.
- Umět pracovat s datem a časem.

Klíčová slova této kapitoly:

pole, array, print_r, count, in_array, array_search, array_keys, datum a čas, date, time, mktime, getdate.

Doba potřebná ke studiu: 2 hodiny



Průvodce studiem

Studium této kapitoly není náročné. Jedná se kapitolu, která studenta seznamuje v dalšími předdefinovanými funkcemi v jazyce PHP. Úvod kapitoly se zabývá prací s polem – vytvoření a procházení pole, vyhledávání v poli, řazení polí. Druhá část kapitoly je věnována práci s datem a časem pomocí funkcí a pomocí objektového přístupu.

Další skupinou předdefinovaných funkcí, které jsou dostupné v jazyce PHP, jsou funkce pro práci s pole a funkce pro práci s kalendářním datem a časem. V této kapitole se seznámíme s některými z těchto funkcí. Stejně jako v případě řetězců nebude jejich výčet úplný

10.1 Práce s polem

Datový typ pole patří mezi složené datové typy. Pole v PHP je v podstatě mapou, která spojuje hodnoty (prvky pole) s jejich klíči (indexy). Indexy mohou být tvořeny řadou čísel, kdy první prvek má index `0`, druhý index `1` atd. Tento způsob reprezentace pole je typický pro jazyky jako C/C++ nebo Java. Kromě toho mohou být indexy prvků tvořeny nespojitou řadou čísel a nebo textovými řetězci. Pokud jsou indexy tvořeny textovými řetězci, hovoříme o asociativním poli. Jako prvky pole můžeme použít i pole, tím vytvoříme dvourozměrné pole.



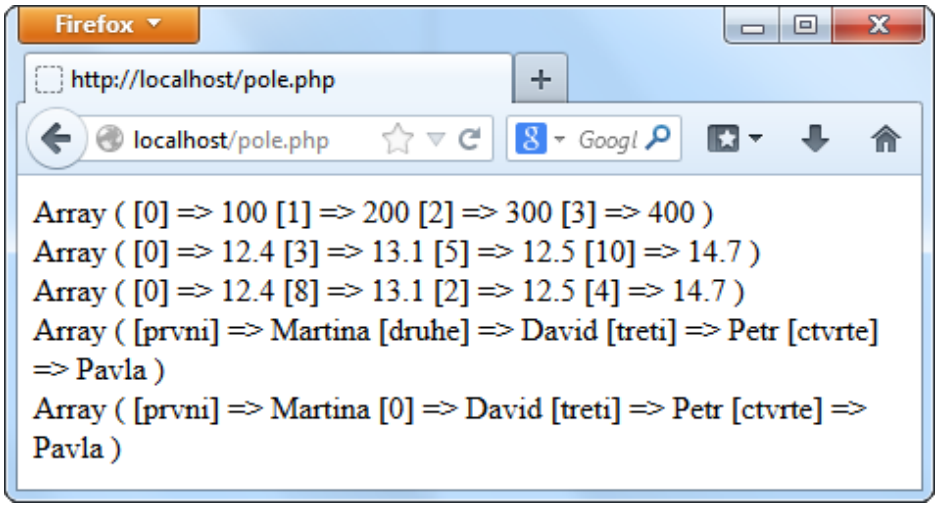
První funkcí, která souvisí s poli, je funkce `array()`, která vytvoří pole a vrátí jeho instanci. Vytvoření pole je možné několika způsoby a jedním z nich je použití této funkce. Jako parametry funkce uvádíme prvky pole. První možností je uvedení prvků pole oddělených čárkami. V takovém případě je

vytvořeno spojitě indexované pole, kdy první prvek pole má index `0`. Druhou možností je uvedení indexů a hodnot, čímž zpravidla vytváříme nespojitě indexované nebo asociativní pole. Jednotlivé prvky jsou opět odděleny čárkami a uvádí se ve tvaru `index => hodnota`. Poslední možností je, že indexy uvedeme pouze u některých prvků. U prvků, u kterých není index uveden, se doplní automaticky. Následující výpis ukazuje různé možnosti použití funkce `array()`. Na obrázku pod výpisem je zobrazen výsledek zobrazený v okně prohlížeče. K vytištění pole je použita funkce `print_r()`, která vytiskne pole v čitelné podobě – vytiskne prvky pole a k nim i jejich indexy.

Výpis 10.1 Ukázka použití funkce array().

```
<?php
  // spojite indexovane pole
  $pole1 = array(100, 200, 300, 400);
  // nespojite indexovane pole
  $pole2 = array(0 => 12.4, 3 => 13.1, 5 => 12.5, 10
   => 14.7);
  // nespojite indexovane pole
  $pole3 = array(0 => 12.4, 8 => 13.1, 2 => 12.5, 4
   => 14.7);
  // asociativni pole
  $pole4 = array("prvni" => "Martina", "druhe" =>
   "David", "treti" => "Petr", "ctvrte" => "Pavla");
  // asociativni pole
  $pole5 = array("prvni" => "Martina", "David",
   "treti" => "Petr", "ctvrte" => "Pavla");
  print_r($pole1);
  echo "<br>";
  print_r($pole2);
  echo "<br>";
  print_r($pole3);
  echo "<br>";
  print_r($pole4);
  echo "<br>";
  print_r($pole5);
?>
```





Obrázek 10.1 Použití funkce array().



Další funkcí, která je velmi často užitečná při práci s polem, je funkce `count()`. Parametrem funkce je pole a návratovou hodnotou je počet prvků tohoto pole. Použití funkce `count()` se hodí zejména v případě, že chceme projít spojitě indexované pole – potom k průchodu polem můžeme použít cyklus `for`. Kromě toho je v PHP k dispozici funkce `sizeof()`, která je aliasem funkce `count()`.

V případě spojitě indexovaných polí můžeme k procházení pole použít cyklus `for`. Pokud ale pracujeme s nespojitě indexovaným nebo asociativním polem, můžeme k procházení pole použít sadu funkcí. Každé pole má interní ukazatel na aktuální prvek pole, který je po vytvoření pole nastaven na první prvek pole.

K získání hodnoty aktuálního prvku slouží funkce `current()`, případně její alias – funkce `pos()`. Parametrem funkcí je pole, ze kterého chceme získat aktuální prvek. Podobnou funkčnost má funkce `each()`, jejímž parametrem je také pole. Funkce vrací nejen hodnotu aktuálního prvku, ale i jeho index a to ve formě pole, které má čtyři prvky. Indexy těchto prvků jsou `0`, `1`, `key` a `value`. Prvky s indexy `0` a `key` obsahují název indexu aktuální prvku. Prvky s indexy `1` a `value` obsahují jeho hodnotu. Kromě toho se funkce posune na následující prvek pole. Pokud se posunem interní ukazatel dostane mimo rozsah pole, vrátí funkce `each()` hodnotu `false`.

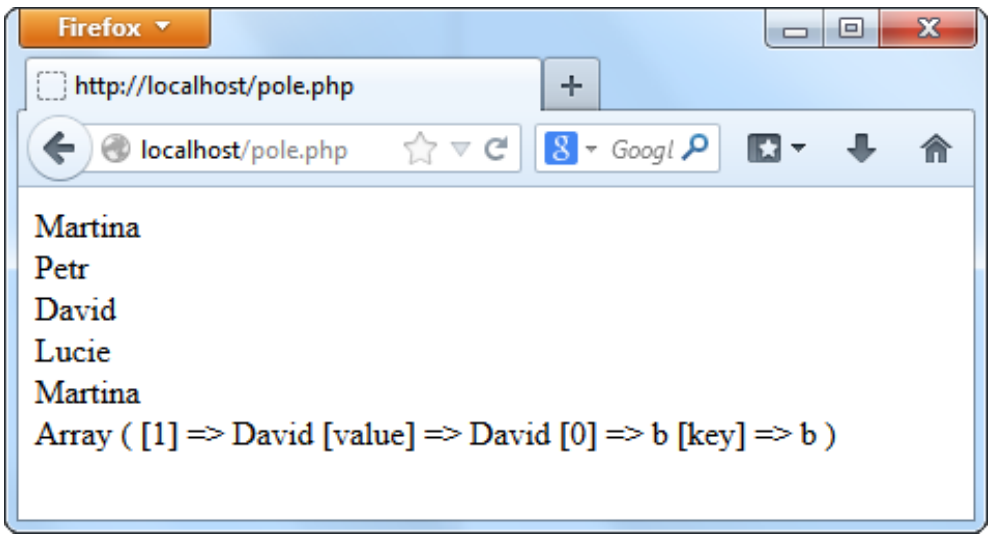
K procházení pole můžeme použít funkce `next()` a `prev()`. Obě funkce mají jako parametr pole. Funkce `next()` se posune na následující prvek pole od aktuální pozice a vrátí jeho hodnotu. Pokud je aktuální pozicí poslední prvek a posunem se dostaneme mimo rozsah pole, vrátí funkce hodnotu

`false`. Funkce `prev()` funguje úplně stejně, ale posouvá se na předchozí prvek od aktuální pozice.

Funkce `reset()` slouží k posunu interního ukazatele pole na první prvek, funkce `end()` slouží k posunu na poslední prvek. Pokud je pole prázdné, vrací obě funkce hodnotu `false`.

Výpis 10.2 Procházení polem promocí funkcí.

```
<?php
  $pole = array("a" => "Martina", "b" => "David", "c"
   => "Petr", "d" => "Pavla", "e" => "Radim", "f" =>
   "Lucie");
  $prvek = current($pole);
  echo $prvek . "<br>";
  $prvek = next($pole);
  $prvek = next($pole);
  echo $prvek . "<br>";
  $prvek = prev($pole);
  echo $prvek . "<br>";
  $prvek = end($pole);
  echo $prvek . "<br>";
  $prvek = reset($pole);
  echo $prvek . "<br>";
  $prvek = next($pole);
  $funkce_each = each($pole);
  print_r($funkce_each);
?>
```



Obrázek 10.2 Ukázka procházení polem pomocí funkcí.



Procházení polem je jednou z nejdůležitějších činností při práci s polem. Kromě toho ale občas potřebujeme i další činnosti – vyhledávat v poli, řadit pole atd. Ke zjištění, zda se v poli vyskytuje určitá hodnota, je určena funkce `in_array()`. Funkce má dva parametry, prvním parametrem je hledaná hodnota, druhým parametrem je pole, ve kterém se hledá. U této funkce je možné použít ještě třetí, volitelný, parametr, kterým se určuje, zda se při hledání v poli kontroluje i shoda datového typu. Funkce `in_array()` vrací `true`, pokud je hledaná hodnota nalezena, jinak vrací `false`. Funkce `in_array()` je case senzitivní. Podobnou funkcí je funkce `array_search()`, která má stejné parametry jako funkce `in_array()`. Funkce nevrací logickou hodnotu, ale index prvku, jehož hodnota odpovídá hledané hodnotě. Pokud není zadaná hodnota nalezena v poli, vrací funkce hodnotu `false`.

Funkce `array_key_exists()` zjišťuje, zda se v poli nachází zadaný index (klíč). Prvním parametrem funkce je hledaný index, druhým parametrem je pole, ve kterém se hledá. Funkce vrací logickou hodnotu `true`, pokud je index nalezen, jinak vrací `false`. Funkce `array_keys()` vrací ve formě pole všechny klíče v zadaném poli. Pole vstupuje do funkce jako parametr. Pomocí dalších volitelných parametrů můžeme určit, že se vrátí pouze některé z indexů. Podobnou funkcí je funkce `array_values()`, která vrací všechny hodnoty zadaného pole. Jediným parametrem funkce je pole, se kterým se pracuje.

Výpis 10.3 Vyhledávání v poli.



```
<?php
  $pole = array("a" => "Martina", "b" => "David", "c"
   => "Petr", "d" => "Pavla", "e" => "Radim", "f" =>
   "Lucie");

  if (in_array("Martina", $pole))
    echo "'Martina' se nachazi v poli<br>";
  else
    echo "'Martina' se nenachazi v poli<br>";

  if (in_array("pavla", $pole))
    echo "'pavla' se nachazi v poli<br>";
  else
    echo "'pavla' se nenachazi v poli<br>";

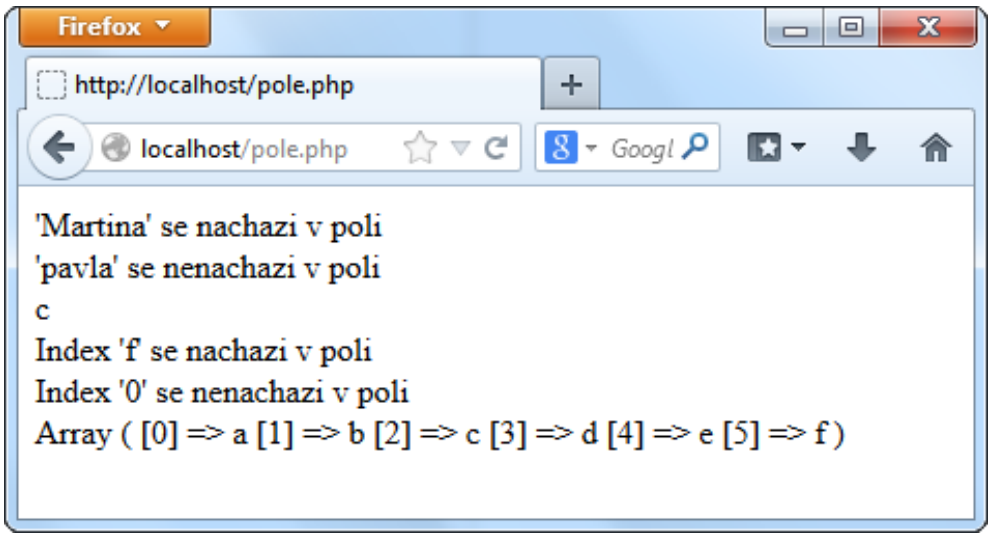
  $index = array_search("Petr", $pole);
```

```
  echo "$index<br>";

  if (array_key_exists("f", $pole))
    echo "Index 'f' se nachazi v poli<br>";
  else
    echo "Index 'f' se nenachazi v poli<br>";

  if (array_key_exists("0", $pole))
    echo "Index '0' se nachazi v poli<br>";
  else
    echo "Index '0' se nenachazi v poli<br>";

  $indexy = array_keys($pole);
  print_r($indexy);
?>
```



Obrázek 10.3 Vyhledávání v poli.

Seřazení pole je možné pomocí funkce `sort()`. Povinným parametrem je pole, které se řadí. Volitelným parametrem je nastavení řazení – porovnání hodnot jako čísel nebo jako řetězců atd. Funkce vrací hodnotu `true`, pokud je řazení úspěšné, jinak vrací hodnotu `false`. Funkce `sort()` řadí prvky od nejmenšího k největšímu. K seřazení od největšího prvku po nejmenší slouží funkce `rsort()`. Parametry jsou stejné jako u funkce `sort()`. Funkce `ksort()` slouží k vzestupnému seřazení pole podle jeho indexů. Sestupné seřazení pole podle indexů je možné pomocí funkce `krsort()`. Obě funkce vrací `true`, pokud je řazení úspěšné, jinak vrací `false`.

Kromě uvedených funkcí, obsahuje jazyk PHP mnoho dalších funkcí pro práci s polem. Pomocí těchto funkcí můžeme například vypočítat součet nebo součin prvků pole (samozřejmě pouze v případě, že prvky pole jsou tvořeny čísly),

spojit více polí dohromady, vytvořit pole obsahující prvky v opačném pořadí, určit jedinečné hodnoty v poli a počet jejich výskytů atd.

10.2 Práce s datem a časem

Při práci s kalendářním datem a časem můžeme použít dva přístupy. První přístup je procedurální, kdy používáme funkce pro datum a čas. Druhý přístup je objektový, k práci s datem a časem používáme třídu `DateTime`.



V procedurálním přístupu jsou základními funkcemi `mktime()`, `date()` a `time()`. Funkce `time()` nemá žádné parametry a vrací celé číslo – aktuální Unixové časové razítko. Časové razítko udává počet sekund od počátku. Jako počátek se považuje datum 1.1.1970 0:0:0 GMT.

Pomocí funkce `mktime()` můžeme pomocí parametrů zadat požadované datum a požadovaný čas. Parametry se zadávají ve formě celých čísel v pořadí hodiny, minuty, sekundy, měsíc, den, rok. Rok se zadává pomocí dvou nebo čtyř číslic. Ostatní parametry se zadávají relativně k nadřazené části data. Nadřazenou částí data je vždy nejbližší větší časový úsek. Roky jsou nadřazeny měsícům, měsíce dnům, dny hodinám atd. Pokud tedy například zadáme v případě měsíce hodnoty `1` až `12`, nastavíme tím příslušný měsíc. Pokud zadáme hodnotu menší než `1`, posuneme datum do předchozího roku než bylo zadáno. Hodnota `0` znamená prosinec předchozího roku, hodnota `-1` listopad atd. Pokud zadáme hodnotu větší než `12`, posouváme datum do následujícího roku. V případě minut jsou nadřazenou částí data hodiny. Hodnoty `0` až `59` nastaví čas na příslušnou minutu. Hodnoty menší než `0` posunou čas do předchozí hodiny, hodnoty větší než `59` do následující hodiny. Funkce `mktime()` vrací celé číslo – Unixové časové razítko odpovídající zadanému datu a času.

Funkce `date()` slouží k naformátování data a času. Povinným parametrem je řetězec, pomocí něhož určujeme formát data a času. Nepovinným parametrem celé číslo udávající Unixové časové razítko. Pokud zadáme pouze první parametr, pro formátování se použije aktuální datum a čas serveru. Zadáním druhého parametru můžeme naformátovat libovolné jiné datum. Formát data a času se určuje pomocí formátovacích znaků, mezi které můžeme vložit oddělovač (mezera, lomítko, tečka atd.). Popis nejdůležitějších formátovacích znaků je uveden v následující tabulce.

Tabulka 10.1 Formátovací znaky pro datum a čas.



Znak	Význam	Hodnoty
d	Den v měsíci zobrazený pomocí dvou číslic, s nulou na začátku.	01 až 31
D	Třípísmenná textová zkratka dne v týdnu.	Mon (pondělí) až Sun (neděle)
j	Den v měsíci zobrazený bez nuly na začátku.	1 až 31
l	Malé písmeno „L“. Plný název dne v týdnu.	Sunday (neděle) až Saturday (sobota)
N	Číselná reprezentace dne v týdnu dle normy ISO-8601.	1 (pondělí) až 7 (neděle)
w	Číselná reprezentace dne v týdnu dle anglického vyjádření.	0 (neděle) až 6 (sobota)
z	Den v roce, začíná od nuly	0 až 365
W	Číslo týdnu v roce dle normy ISO-8601.	0 až 52
F	Plný název měsíce.	January (leden) až December (prosinec)
m	Číselná reprezentace měsíce zobrazená pomocí dvou číslic.	01 až 12
M	Třípísmenná zkratka měsíce	Jan (leden) až Dec (prosinec)
n	Číselná reprezentace měsíce bez nuly na začátku	1 až 12
t	Počet dnů v zadaném měsíci.	28 až 31
L	Zjištění, zda je přestupný rok.	1 (ano) nebo 0 (ne)
Y	Plné vyjádření roku – čtyři číslice.	např. 1981 nebo 2013
y	Vyjádření roku pomocí dvou číslic (poslední dvě číslice)	např. 81 nebo 13
a	Určení zda se jedná o dopoledne nebo odpoledne, malá písmena.	am nebo pm
A	Určení, zda se jedná o dopoledne nebo odpoledne, velká písmena	AM nebo PM
g	Hodiny, 12-hodinový formát, bez nuly na začátku.	1 až 12
G	Hodiny, 24-hodinový formát, bez nuly na začátku.	0 až 23
h	Hodiny, 12-hodinový formát, s nulou na začátku.	01 až 12

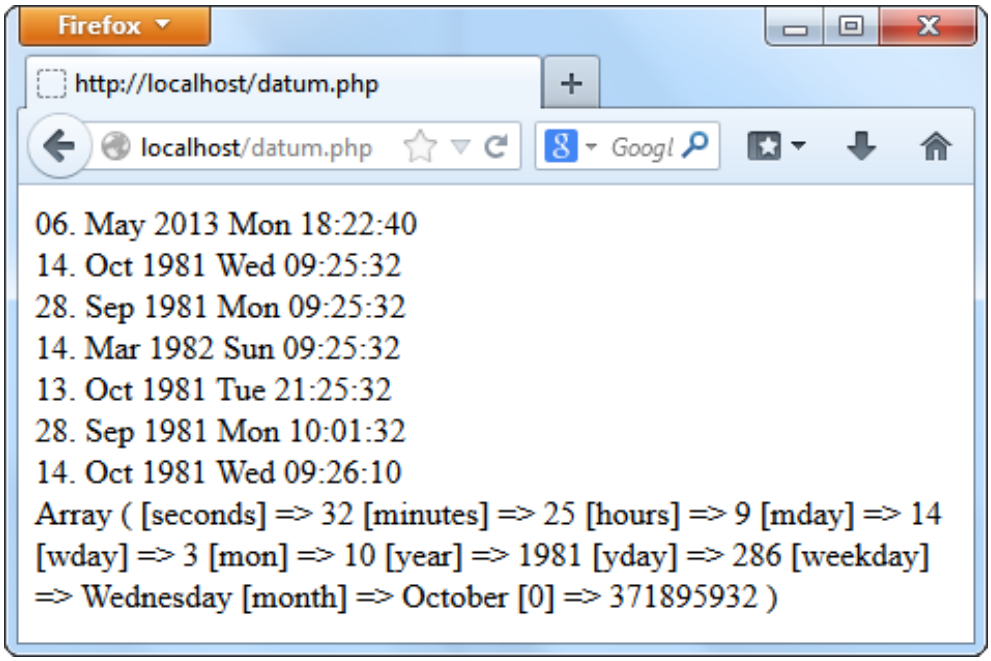
H	Hodiny, 24-hodinový formát, s nulou na začátku.	00 až 23
i	Minuty, s nulou na začátku.	00 až 59
s	Sekundy, s nulou na začátku.	00 až 59

Funkce `getdate()` vrací asociativní pole, které reprezentuje datum. Pokud funkci zavoláme bez parametrů, je použito aktuální datum a aktuální čas serveru. Volitelným parametrem je zadání časového razítka, které reprezentuje jakékoliv jiné datum.

Výpis 10.4 Práce s datem a časem.



```
<?php
  // aktualni datum a cas
  $dnes = date("d. M Y D H:i:s");
  echo "$dnes<br>";
  // 14. rijna 1981 9 hod 25 min 32 sek
  $dnes = date("d. M Y D H:i:s", mktime(9, 25, 32,
   10, 14, 1981));
  echo "$dnes<br>";
  // posun dne
  $dnes = date("d. M Y D H:i:s", mktime(9, 25, 32,
   10, -2, 1981));
  echo "$dnes<br>";
  // posun mesice
  $dnes = date("d. M Y D H:i:s", mktime(9, 25, 32,
   15, 14, 1981));
  echo "$dnes<br>";
  // posun hodin
  $dnes = date("d. M Y D H:i:s", mktime(-3, 25, 32,
   10, 14, 1981));
  echo "$dnes<br>";
  // posun minut
  $dnes = date("d. M Y D H:i:s", mktime(9, 61, 32,
   10, -2, 1981));
  echo "$dnes<br>";
  // posun sekund
  $dnes = date("d. M Y D H:i:s", mktime(9, 25, 70,
   10, 14, 1981));
  echo "$dnes<br>";
  // pouziti funkce getdate()
  $dnes = getdate(mktime(9, 25, 32, 10, 14, 1981));
  print_r($dnes);
?>
```



Obrázek 10.4 Práce s datem a časem.



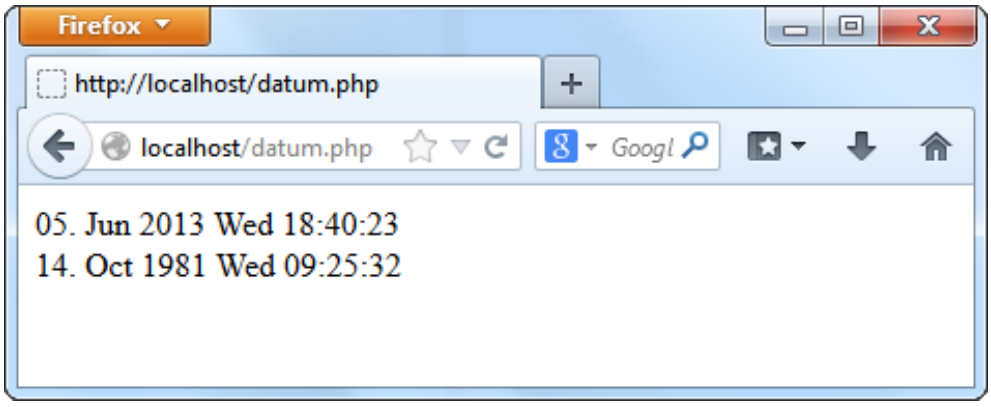
Práce s datem a časem pomocí objektového přístupu je možná pomocí třídy `DateTime`. Vytvořením nové instance této třídy vytvoříme objekt představující datum a čas. Pokud použijeme konstruktor bez parametrů, použije se aktuální datum a čas. Jiné datum a čas můžeme vytvořit jeho zadáním při vytváření objektu. Třída `DateTime` obsahuje metody, které mají stejnou funkčnost jako při použití procedurálního přístupu. K naformátování data slouží metoda `format()`, k nastavení data slouží metoda `setDate()`, k nastavení času metoda `setTime()` a k nastavení časového razítka metoda `setTimestamp()`.

Výpis 10.5 Použití třídy DateTime.



```
<?php
  // aktualni datum a cas
  $dnes = new DateTime();
  echo $dnes->format("d. M Y D H:i:s") . "<br>";

  // zmena data a casu
  $dnes->setDate(1981, 10, 14);
  $dnes->setTime(9, 25, 32);
  echo $dnes->format("d. M Y D H:i:s") . "<br>";
?>
```

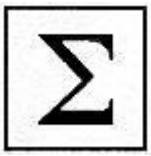


Obrázek 10.5 Použití třídy DateTime.



Kontrolní otázky:

1. Jakým způsobem je možné indexovat prvky pole?
2. Jaké funkce je možné použít pro vyhledávání v poli?
3. K čemu slouží funkce `mktime()`?
4. K čemu slouží funkce `getdate()`?



Shrnutí obsahu kapitoly

V této kapitole jste se seznámili s předdefinovanými funkcemi v PHP, které slouží pro práci s polem a kalendářním datem a časem. Jazyk PHP umožňuje vytvářet spojitě nebo nespojitě indexovaná pole nebo asociativní pole. Při práci s polem můžeme použít funkce pro procházení polem, vyhledávání v poli nebo řazení pole. Práce s kalendářním datem a časem je možné procedurálním způsobem (pomocí funkcí) nebo objektově (pomocí třídy DateTime). Zadané datum je možné naformátovat různými způsoby pomocí speciálních formátovacích znaků. Jazyk PHP obsahuje mnoho funkcí, jejich popis v této kapitole není úplný, jsou zmíněny jen některé z funkcí.